

SCRIPT ORAL — UNE VIE DE FOURMI

Romain • Lisa • Yannis

Équipe : Romain • Lisa • Yannis

Durée visée : 9 à 10 minutes

Objectif : vous pouvez quasiment le lire tel quel. Gardez un ton naturel : regardez le jury à chaque fin de phrase et utilisez les diapositives comme support, pas comme prompteur.

Répartition

Personne	Partie	Temps	Mission
Romain	Diapos 1 à 4	~3 min	Accroche, règles, modélisation en graphe
Lisa	Diapos 5 à 8	~3 min	Pourquoi le plus court chemin ne suffit pas, algorithme optimal
Yannis	Diapos 9 à 12	~3 min	Code, démonstration, tests, conclusion

Plan B si la soutenance est courte : coupez la diapo 8 et résumez les tests de la diapo 11 en une phrase. Vous retombez à environ 7 minutes.

Diapo 1 — Une vie de fourmi — Romain — 35 à 45 s

Bonjour. Nous sommes Romain, Lisa et Yannis, et notre projet s'appelle « Une vie de fourmi ».

Le problème paraît simple : faire passer toutes les fourmis du vestibule Sv au dortoir Sd. En réalité, il faut organiser plusieurs déplacements en parallèle, respecter la capacité des salles et surtout garantir qu'on utilise le nombre minimum d'étapes.

Notre objectif a donc été de construire une solution qui soit à la fois correcte, optimale et facile à visualiser.

Mise en scène : commencer sans lire l'écran. À la fin de « minimum d'étapes », faire une courte pause.

À retenir : parallèle, capacités, optimal.

Diapo 2 — Le problème en une image — Romain — 45 s

On peut comparer la fourmilière à un réseau de métro. Les salles sont les stations, les tunnels sont les lignes et les fourmis sont les passagers.

Le piège, c'est que certaines stations n'acceptent qu'une seule fourmi. Donc mettre tout le monde sur le chemin le plus court peut créer un embouteillage.

Il faut parfois répartir les fourmis sur plusieurs chemins pour que la dernière arrive plus tôt.

Mise en scène : pointer Sv, puis Sd, puis les deux chemins du graphe.

À retenir : le problème n'est pas « trouver un chemin », mais « organiser un trafic ».

Diapo 3 — Les règles à respecter — Romain — 50 s

Nous avons repris les règles du sujet sans les simplifier. À chaque étape, une fourmi peut attendre ou se déplacer vers une salle voisine.

Les salles intermédiaires ont une capacité de 1 par défaut, sauf quand une capacité X est indiquée. Le vestibule et le dortoir ne sont pas limitants.

Et surtout, les mouvements d'une même étape sont simultanés : une fourmi peut entrer dans une salle si celle qui l'occupait est en train d'en sortir.

Mise en scène : insister sur « simultanés » : c'est une règle souvent oubliée.

À retenir : capacité à chaque instant, mouvements simultanés, tunnels = adjacence.

Diapo 4 — Modélisation en graphe — Romain — 50 s

Pour représenter la fourmilière, nous utilisons un graphe non orienté avec NetworkX. Une salle devient un sommet et un tunnel devient une arête.

On stocke aussi la capacité de chaque salle. Cette représentation nous donne directement les voisins d'une salle, les chemins et la matrice d'adjacence.

À partir de là, Lisa va expliquer pourquoi un simple plus court chemin ne suffit pas pour trouver la meilleure solution globale.

Mise en scène : transitionner clairement vers Lisa.

À retenir : sommet = salle, arête = tunnel, capacité = contrainte.

Diapo 5 — Pourquoi le plus court chemin ne suffit pas — Lisa — 45 à 55 s

Si on calcule seulement le plus court chemin entre Sv et Sd, on résout le problème d'une seule fourmi. Mais nous en avons F.

Avec plusieurs fourmis, le facteur décisif devient le débit : combien de fourmis peuvent traverser le réseau en parallèle sans dépasser la capacité des salles ?

Deux chemins légèrement plus longs peuvent être meilleurs qu'un seul chemin très court mais saturé.

Mise en scène : faire l'analogie « route rapide à une voie contre deux routes ».

À retenir : distance seule $\neq$ temps total de la colonie.

Diapo 6 — Notre idée : déplier le temps — Lisa — 65 à 75 s

Notre solution est de transformer la fourmilière en un graphe temporel.

Pour tester une durée T , on crée une copie de chaque salle pour chaque instant : temps 0, temps 1, jusqu'à temps T .

Une arête entre une salle au temps t et une salle voisine au temps $t+1$ représente un déplacement. Une arête vers la même salle au temps suivant représente l'attente.

Enfin, chaque salle est séparée en une entrée et une sortie, avec une capacité entre les deux. C'est ce qui impose exactement le nombre maximal de fourmis présentes dans cette salle.

Mise en scène : pointer la mini-frise temporelle de gauche à droite.

À retenir : le graphe temporel encode toutes les décisions possibles et toutes les capacités.

Diapo 7 — Comment garantir le minimum — Lisa — 60 à 70 s

On commence par d , la longueur du plus court chemin entre S_v et S_d . Avant d étapes, même une seule fourmi ne peut pas arriver, donc c'est notre borne basse.

On teste ensuite T égal à d , puis d plus 1, d plus 2, et ainsi de suite. Pour chaque T , on calcule un flot maximal.

Si le flot maximal vaut F , cela signifie que F trajectoires simultanées respectant toutes les contraintes existent. Le premier T qui fonctionne est forcément le minimum.

C'est la partie importante : on ne se contente pas d'une heuristique, on obtient une optimalité garantie.

Mise en scène : dire lentement « premier T qui fonctionne = minimum ».

À retenir : c'est la preuve d'optimalité à connaître par cœur.

Diapo 8 — Rendre la solution lisible — Lisa — 35 à 45 s

Une fois l'horizon minimal trouvé, nous recalculons un flot de coût minimal. Le but n'est pas de changer le nombre d'étapes, mais de favoriser des arrivées plus tôt au dortoir et d'éviter des trajectoires inutilement compliquées.

Ensuite, le flot est décomposé en trajectoires individuelles f_1 , f_2 , f_3 , etc., ce qui nous permet d'afficher exactement les étapes demandées dans le sujet.

Mise en scène : peut être coupée si le temps est serré.

À retenir : max-flow trouve la faisabilité ; min-cost améliore la lisibilité à horizon déjà optimal.

Diapo 9 — Architecture du code — Yannis — 55 à 65 s

Le projet est séparé en trois fichiers principaux.

`ants.py` contient le modèle de la fourmilière, le parseur et l'algorithme de résolution.

`main.py` gère l'entrée, lance la résolution et exporte les résultats.

visualization.py produit le graphe, les images étape par étape et l'animation GIF.

Cette séparation rend le code plus simple à tester et évite d'avoir toute la logique dans un seul fichier.

Mise en scène : montrer les trois blocs du schéma, pas le code ligne par ligne.

À retenir : modèle/algorithmes — orchestration — visualisation.

Diapo 10 — Démonstration : le cas simple — Yannis — 70 à 85 s

Sur le cas simple du sujet, nous avons 3 fourmis et deux chemins de longueur 2.

À l'étape 1, f1 entre dans S1 et f2 dans S2.

À l'étape 2, ces deux fourmis entrent dans Sd pendant que f3 entre dans S1. C'est possible parce que les mouvements sont simultanés.

À l'étape 3, f3 rejoint Sd. On retrouve donc exactement les 3 étapes annoncées dans le sujet.

Le programme génère en plus solution.txt, le graphe, la matrice d'adjacence, toutes les images d'étapes et une animation GIF.

Mise en scène : faire défiler $E1 \rightarrow E2 \rightarrow E3$. Ne pas rester plus de 5 secondes sur chaque image.

À retenir : la démo valide visuellement la règle de simultanéité.

Diapo 11 — Tests et robustesse — Yannis — 45 à 55 s

Nous avons ajouté des tests automatiques pour vérifier plusieurs situations : le cas simple, un tunnel direct, un goulot d'étranglement de capacité 1, une capacité 2 et un graphe sans chemin jusqu'au dortoir.

Le programme vérifie aussi après résolution que chaque déplacement correspond bien à un tunnel et que les capacités ne sont jamais dépassées.

Le parseur accepte plusieurs écritures du nombre de fourmis et les capacités sous la forme $S1\{X\}$.

Mise en scène : afficher « 6 tests OK » comme preuve simple.

À retenir : on teste le résultat, pas seulement l'absence d'erreur Python.

Diapo 12 — Conclusion — Yannis — 35 à 45 s

Pour conclure, nous avons transformé un problème de déplacement de fourmis en problème de graphe et de flot.

Notre solution respecte les contraintes du sujet, produit toutes les visualisations demandées et garantit le nombre minimal d'étapes.

Le principal enseignement est que le chemin le plus court ne suffit pas quand plusieurs agents partagent des ressources limitées : il faut optimiser le trafic global. Merci, nous sommes prêts pour vos questions.

Mise en scène : finir en regardant le jury, puis laisser la diapo Q&R affichée.

Dernière phrase forte : « Le plus court chemin optimise un trajet. Notre algorithme optimise tout le trafic. »

Démonstration live — procédure ultra sûre

- Ouvrir un terminal dans le dossier `uneviedefourmi`.
- Vérifier avant la soutenance : `python -m unittest discover -s tests -v`.
- Lancer : `python main.py inputs/cas_simple.txt`.
- Montrer d'abord la sortie console E1 / E2 / E3.
- Ouvrir ensuite `outputs/cas_simple/graphe.png`, puis `animation.gif`.
- Pour tous les cas : `python main.py inputs` traite tous les `.txt` du dossier.

Phrase de secours si la démo plante :

« La démo locale rencontre un problème d'environnement, mais les sorties générées à l'avance correspondent au même exécutable et les tests automatiques sont validés. Je vous montre le résultat. »

Questions probables — réponses flash

Question	Réponse courte
Pourquoi NetworkX ?	Parce qu'il fournit une représentation claire des graphes et des algorithmes de flot fiables, tout en nous laissant gérer nous-mêmes la modélisation du problème.
Pourquoi pas seulement BFS ?	BFS trouve un plus court chemin pour un trajet individuel. Ici il faut coordonner F fourmis avec des capacités partagées.
Comment prouvez-vous que c'est optimal ?	On teste les horizons dans l'ordre croissant. Le premier T pour lequel un flot de valeur F existe est le premier temps où toutes les trajectoires sont réalisables.
Pourquoi découper une salle en entrée/sortie ?	Une capacité d'arête est facile à imposer dans un réseau de flot. Le découpage transforme donc une capacité de salle en capacité d'arête.
Les fourmis peuvent-elles attendre ?	Oui. Dans le graphe temporel, l'attente est une arête de la salle à elle-même entre t et $t+1$.
Et si deux fourmis veulent la même salle ?	La capacité de la salle limite automatiquement le flot qui peut la traverser à cet instant.
Pourquoi un min-cost après le max-flow ?	L'horizon est déjà optimal. Le coût sert seulement à choisir, parmi les solutions optimales, une solution plus lisible avec des arrivées précoces.
Limite principale ?	Le graphe temporel grossit avec T, le nombre de salles et le nombre de tunnels. C'est un compromis volontaire pour obtenir une garantie d'optimalité.